

Software Design Specification (SDS)

A Document Integrity and Confidentiality System

Version: 1.5

Date: July 22, 2025

Author: Okoth Bernard Wycliffe – 23/08325

Project Type: Security and Forensics Undergraduate Project (BISF 2208)

Table of Contents

1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Audience	5
1.4 Project Constraints.....	5
2. System Overview.....	5
2.1 System Purpose	5
2.2 Key Features.....	5
2.3 System Benefits	6
3. Functional Requirements.....	6
3.1 Core Functional Requirements.....	6
3.2 GUI Functional Requirements	7
3.3 Utility Functional Requirements.....	8
4. Non-Functional Requirements.....	8
4.1 Performance Requirements.....	8
4.2 Security Requirements	8
4.3 Usability Requirements.....	8
4.4 Reliability Requirements.....	9
4.5 Compatibility Requirements	9
5. System Architecture	9
5.1 High-Level Architecture.....	9
5.2 Component Interaction Flow.....	9

5.2 Component Interaction Flow	10
5.3 Module Dependencies	10
6. Detailed Design	11
6.1 Core Module Design	11
6.1.1 Hash Generator Module	11
6.1.2 Encryptor Module	12
6.1.3 Steganography Module	12
6.1.4 Verifier Module	13
6.2 Data Structures	14
6.2.1 Verification Result	14
6.2.2 Processing Status	14
6.3 Algorithm Specifications	15
6.3.1 LSB Steganography Algorithm	15
6.3.2 Text Steganography Algorithm	15
7. User Interface Design	15
7.1 Main Window Layout	15
7.2 Dialog Windows	17
7.2.1 Key Management Dialog	17
7.2.2 Verification Results Dialog	18
7.3 GUI Component Specifications	18
7.3.1 File List Component	18
7.3.2 Progress Indicator Component	19
7.4 User Experience Flow	19
7.4.1 Document Protection Flow	19
7.4.2 Document Verification Flow	20
8. Database Design	20
8.1 File-Based Storage Strategy	20
8.1.1 Configuration Storage	20
8.1.2 Key Storage Format	21
8.1.3 Processing Log Format	21
8.2 Data Persistence Strategy	22
8.2.1 Key Management	22
8.2.2 Audit Trail	22

8.3 File Organization Structure	22
9. Security Design.....	23
9.1 Cryptographic Standards.....	23
9.1.1 Hash Functions	23
9.1.2 Encryption Standards	23
9.1.3 Random Number Generation.....	23
9.2 Key Management Security	23
9.2.1 Key Lifecycle.....	23
9.2.2 Key Storage Security	24
9.2.3 Memory Security.....	24
9.3 Attack Mitigation	24
9.3.1 Common Attack Vectors	24
9.3.2 Security Validation.....	24
9.4 Compliance Considerations	25
9.4.1 Academic Standards	25
9.4.2 Export Control	25
10. Implementation Plan.....	25
10.1 Development Phases.....	25
10.2 Resource Allocation	28
10.3 Risk Management	28
11. Testing Strategy.....	29
11.1 Testing Approach.....	29
11.2 Test Categories	29
11.2.1 Functional Testing	29
11.2.2 Security Testing.....	30
11.2.3 Performance Testing.....	31
11.2.4 Usability Testing.....	31
11.3 Test Data Management	32
11.4 Test Automation.....	32
12. Deployment Plan	33
12.1 Deployment Strategy	33
12.2 Installation Package.....	33
12.3 Installation Process	34

12.4 System Requirements	35
12.5 Configuration Management	36
12.6 Update Mechanism	36
12.7 Uninstallation	37
Appendices.....	38
Appendix A: Glossary	38
Appendix B: References	39
Appendix C: Code Standards	39
Python Style Guide:	39
Appendix D: Academic Compliance.....	39
Project Requirements Fulfillment:.....	39
Learning Objectives Achievement:.....	39

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) document provides a comprehensive technical design for the Document Integrity and Confidentiality System, focused on implementing cryptographic techniques to ensure document authenticity and for detection of tampering.

1.2 Scope

The system will provide:

- Document hash generation using SHA-256
- AES encryption for hash protection
- Steganographic techniques to embed integrity data
- Document verification and tampering detection
- User-friendly GUI interface for all operations

1.3 Audience

- **Primary:** Academic supervisors and project evaluators
- **Secondary:** Potential users interested in document integrity solutions

1.4 Project Constraints

Platform: Windows 10 development environment

IDE: PyCharm Community Edition

Language: Python 3.8+

Timeline: 8 weeks academic semester

Complexity: Suitable for academic document protection

2. System Overview

2.1 System Purpose

The Document Integrity and Confidentiality System ensures document authenticity by:

- Generating cryptographic hashes of documents
- Encrypting these hashes for security
- Embedding encrypted hashes using steganography
- Providing verification mechanisms to detect document tampering

2.2 Key Features

- **Multi-format Support:** PDF, and images
- **Cryptographic Security:** SHA-256 hashing with AES-256 encryption
- **Steganographic Embedding:** LSB method for images, pdf metadata, whitespace for text

- **Intuitive GUI:** Tkinter-based interface for ease of use
- **Batch Processing:** Handle multiple documents simultaneously (future integration)
- **Comprehensive Logging:** Track all operations and errors

2.3 System Benefits

- **Academic Value:** Demonstrates practical cryptography implementation
- **Real-world Application:** Addresses genuine document security needs
- **Skill Development:** Covers GUI development, cryptography, and testing
- **Portfolio Enhancement:** Showcases technical and project management skills

3. Functional Requirements

3.1 Core Functional Requirements

FR-001: Document Hash Generation

Description: Generate SHA-256 hash for uploaded documents

Input: Document file (PDF, PNG)

Process: Calculate cryptographic hash of file contents

Output: 256-bit hash value in hexadecimal format

Priority: High

FR-002: Hash Encryption

Description: Encrypt document hashes using AES-256

Input: SHA-256 hash and encryption key

Process: Apply AES encryption in CBC mode

Output: Encrypted hash data

Priority: High

FR-003: Steganographic Embedding

Description: Hide encrypted hash within document using steganography

Input: Document and encrypted hash

Process:

Text files: Use whitespace manipulation

Images: Use LSB (Least Significant Bit) method

Output: Document with embedded integrity data

Priority: High

FR-004: Document Verification

Description: Verify document integrity by extracting and checking embedded hash

Input: Document with embedded hash

Process: Extract hash, decrypt, and compare with current stored document hash

Output: Verification result (Pass/Fail) with confidence level

Priority: High

FR-005: Key Management

Description: Generate, store, and manage encryption keys

Input: User request for key generation

Process: Generate secure random AES keys

Output: Encrypted key files with secure storage

Priority: Medium

3.2 GUI Functional Requirements

FR-006: File Upload Interface

Description: Provide intuitive file selection mechanism

Input: User file selection

Process: Validate file format and size

Output: Loaded file ready for processing

Priority: High

FR-007: Progress Tracking

Description: Show real-time progress for long operations

Input: Operation status updates

Process: Update progress bar and status messages

Output: Visual feedback to user

Priority: Medium

FR-008: Results Display

Description: Present verification results clearly

Input: Verification outcome data

Process: Format results with appropriate visual indicators

Output: User-friendly results display

Priority: High

3.3 Utility Functional Requirements

FR-009: Logging System

Description: Record all system operations and errors

Input: System events and error conditions

Process: Format and store log entries

Output: Structured log files for debugging

Priority: Medium

FR-010: Batch Processing (future integration)

Description: Process multiple documents in sequence

Input: List of documents to process

Process: Apply protection/verification to each document

Output: Batch processing results summary

Priority: Low

4. Non-Functional Requirements

4.1 Performance Requirements

- **Response Time:** GUI operations complete within 2 seconds
- **Throughput:** Process up to 10MB documents efficiently
- **Scalability:** Handle up to 100 documents in batch mode (future integration)
- **Memory Usage:** Maximum 512MB RAM during operation

4.2 Security Requirements

- **Encryption Standard:** AES-256 encryption for all sensitive data
- **Key Security:** Secure key generation using OS random sources
- **Data Protection:** No plaintext storage of sensitive information
- **Access Control:** File system permissions for key storage

4.3 Usability Requirements

- **Learning Curve:** New users operational within 10 minutes
- **Error Handling:** Clear error messages with suggested solutions

- **Accessibility:** Standard Windows accessibility compliance
- **Help System:** Integrated help and user manual

4.4 Reliability Requirements

- **Availability:** 99.9% uptime during operation
- **Error Recovery:** Graceful handling of unexpected conditions
- **Data Integrity:** No data corruption during processing
- **Backup:** Automatic backup of critical configuration

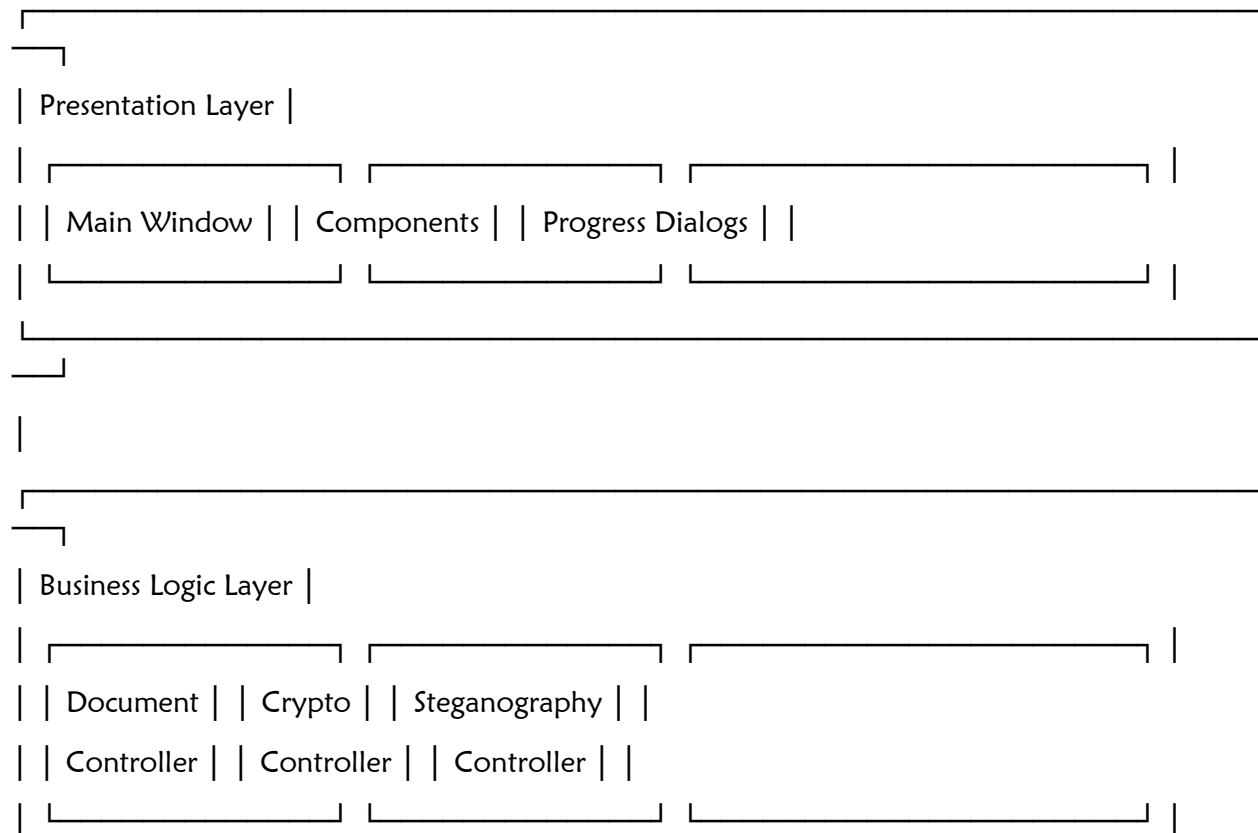
4.5 Compatibility Requirements

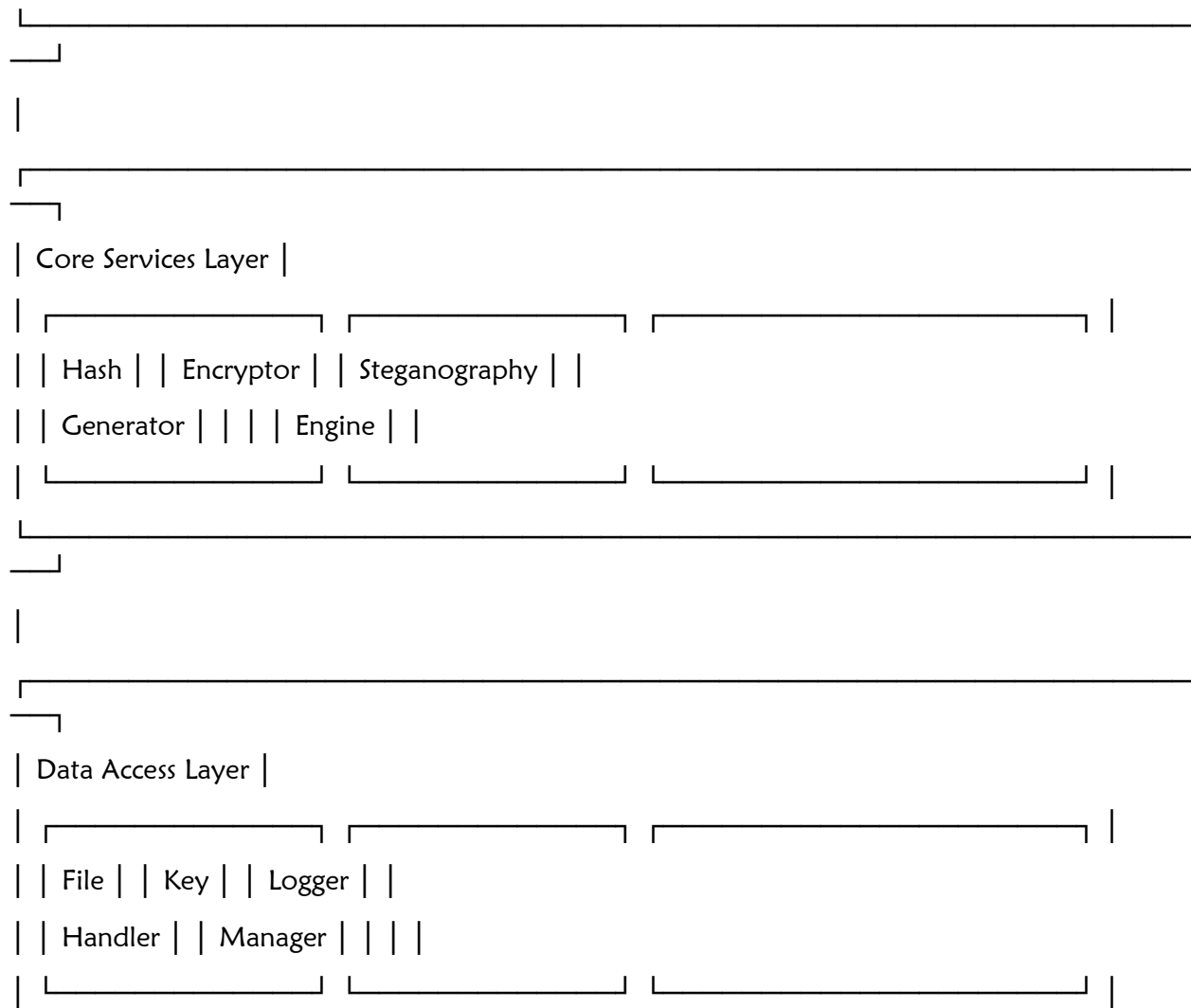
- **Operating System:** Windows 10/11 (primary), Linux/macOS (secondary)
- **Python Version:** 3.8 or higher
- **File Formats:** PDF, PNG
- **Dependencies:** Minimal external library requirements

5. System Architecture

5.1 High-Level Architecture

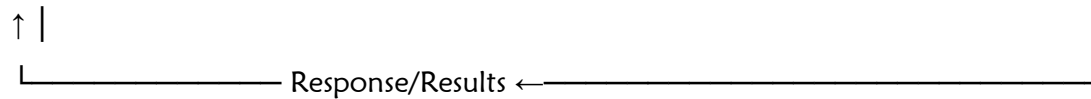
5.2 Component Interaction Flow





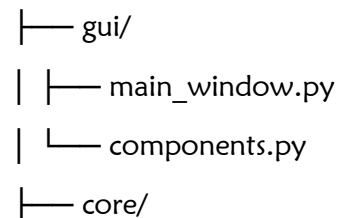
5.2 Component Interaction Flow

User Action → GUI Layer → Business Logic → Core Services → Data Layer



5.3 Module Dependencies

main.py



```
| |— hash_generator.py
| |— encryptor.py
| |— steganography.py
| |— verifier.py
| |— utils/
| |— file_handler.py
| |— logger.py
| |— helpers.py
```

6. Detailed Design

6.1 Core Module Design

6.1.1 Hash Generator Module

python

```
class HashGenerator:
```

```
"""
```

```
Responsible for generating cryptographic hashes of documents
```

```
"""
```

```
def __init__(self, algorithm='sha256'):
```

```
    self.algorithm = algorithm
```

```
    self.hasher = hashlib.new(algorithm)
```

```
def generate_file_hash(self, file_path: str) -> str:
```

```
    """Generate hash for file contents"""
```

```
def generate_text_hash(self, text: str) -> str:
```

```
    """Generate hash for text content"""
```

```
def verify_hash(self, data: bytes, expected_hash: str) -> bool:
```

```
    """Verify data against expected hash"""
```

Key Methods:

generate_file_hash() : Read file in chunks, compute SHA-256
generate_text_hash() : Hash text content directly
verify_hash() : Compare computed hash with expected value

6.1.2 Encryptor Module

```
python
class AESEncryptor:
    """
    Handles AES encryption and decryption operations
    """

    def __init__(self, key_size=32):
        self.key_size = key_size
        self.block_size = 16

    def generate_key(self) -> bytes:
        """Generate secure random key"""

    def encrypt(self, data: bytes, key: bytes) -> bytes:
        """Encrypt data using AES-256-CBC"""

    def decrypt(self, encrypted_data: bytes, key: bytes) -> bytes:
        """Decrypt data using AES-256-CBC"""
```

Security Features:

- PKCS7 padding for block alignment
- Random IV generation for each encryption
- Secure key derivation using PBKDF2

6.1.3 Steganography Module

```
python
class SteganographyEngine:
    """
```

Implements steganographic techniques for different file types

```
"""
```

```
def embed_in_text(self, text: str, data: bytes) -> str:
```

```
    """Hide data in text using whitespace manipulation"""
```

```
def extract_from_text(self, text: str) -> bytes:
```

```
    """Extract hidden data from text"""
```

```
def embed_in_image(self, image_path: str, data: bytes) -> Image:
```

```
    """Hide data in image using LSB method"""
```

```
def extract_from_image(self, image_path: str) -> bytes:
```

```
    """Extract hidden data from image"""
```

Techniques:

- **Text:** Zero-width characters, whitespace manipulation
- **Images:** LSB modification in RGB channels
- **Capacity:** Calculate maximum data capacity per file type

6.1.4 Verifier Module

python

```
class DocumentVerifier:
```

```
    """
```

Orchestrates the document verification process

```
    """
```

```
def __init__(self):
```

```
    self.hash_gen = HashGenerator()
```

```
    self.encryptor = AESEncryptor()
```

```
    self.stego = SteganographyEngine()
```

```
def protect_document(self, file_path: str, key: bytes) -> str:
```

```
    """Apply integrity protection to document"""
```

```
def verify_document(self, file_path: str, key: bytes) -> VerificationResult:
```

```
    """Verify document integrity"""
```

6.2 Data Structures

6.2.1 Verification Result

```
python
@dataclass
class VerificationResult:
    is_authentic: bool
    confidence_level: float
    original_hash: str
    current_hash: str
    timestamp: datetime
    file_path: str
    error_message: Optional[str] = None
```

6.2.2 Processing Status

```
python
@dataclass
class ProcessingStatus:
    stage: str
    progress_percent: int
    current_file: str
    total_files: int
    completed_files: int
    estimated_time_remaining: int
```

6.3 Algorithm Specifications

6.3.1 LSB Steganography Algorithm

- For each pixel in image:
- For each color channel (R, G, B):
- If data bits remaining:
- Replace LSB of channel with data bit
- Increment data bit pointer

Else:

- Break embedding process

6.3.2 Text Steganography Algorithm

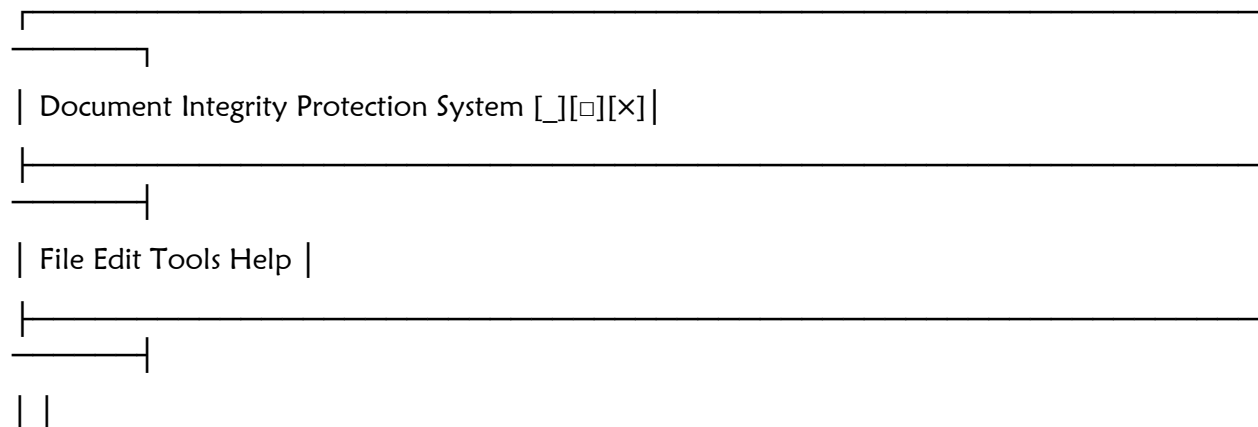
- For each character in text:
- If data bits remaining:
- Add zero-width character based on data bit:
- Bit 0: Zero-width space (U+200B)
- Bit 1: Zero-width non-joiner (U+200C)
- Increment data bit pointer

Else:

- Continue with original text

7. User Interface Design

7.1 Main Window Layout



Select Protect Verify			
Documents Documents Documents			
Selected Files:			
important_document.pdf certificate.docx signature.png			
Processing Status:			
Ready for operation... 100%			
Results:			
Status	File	Hash	Notes

✓	Pass	document.pdf	a1b2c3d4...	OK
✗	Fail	certificate.docx	e5f6g7h8...	Error

7.2 Dialog Windows

7.2.1 Key Management Dialog

Encryption Key Management [X]	
Current Key: [Browse...] [Generate New]	
Key Strength: ● AES-256 ○ AES-128	
Key File: encryption_key.key	
Key fingerprint:	
SHA256: a1b2c3d4e5f6g7h8...	
[Save Key] [Load Key] [Cancel]	

7.2.2 Verification Results Dialog

Document Verification Results [X]	
File: important_document.pdf	
Status: ✓ AUTHENTIC	
Confidence: 100%	
Original Hash:	
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0	
Current Hash:	
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0	
Protection Date: 2025-06-29 14:30:15	
Verification Date: 2025-06-29 16:45:22	
[Save Report] [Close]	

7.3 GUI Component Specifications

7.3.1 File List Component

python

```
class FileListWidget(tk.Frame):
```

```
"""
```

Custom widget for displaying selected files with status

```
"""
```

```
def __init__(self, parent):
```

```

super().__init__(parent)
self.files = []
self.setup_ui()
def add_file(self, file_path: str):
    """Add file to list with icon based on type"""
def update_status(self, file_path: str, status: str):
    """Update processing status for specific file"""
def clear_files(self):
    """Remove all files from list"""

```

7.3.2 Progress Indicator Component

```

python
class ProgressIndicator(tk.Frame):
    """
    Custom progress indicator with status text
    """
    def __init__(self, parent):
        super().__init__(parent)
        self.progress_var = tk.DoubleVar()
        self.status_var = tk.StringVar()
        self.setup_ui()
    def update_progress(self, percent: float, status: str):
        """Update progress bar and status text"""
    def reset(self):
        """Reset progress to initial state"""

```

7.4 User Experience Flow

7.4.1 Document Protection Flow

1. User clicks "Select Documents"
2. File dialog opens for multi-selection
3. Selected files appear in file list
4. User clicks "Protect Documents"
5. Key management dialog appears
6. User selects/generates encryption key
7. Progress indicator shows processing status
8. Results display shows protection outcome
9. Protected files saved to output directory

7.4.2 Document Verification Flow

1. User clicks "Verify Documents"
2. File dialog opens for protected files
3. Selected files appear in file list
4. User provides encryption key
5. Progress indicator shows verification status
6. Results dialog shows detailed verification
7. Report can be saved for documentation

8. Database Design

8.1 File-Based Storage Strategy

The system uses file-based storage with structured formats:

8.1.1 Configuration Storage

config.json

json

```
{  
  "application": {  
    "name": "Document Integrity Protection System",  
    "version": "1.0.0",  
    "author": "Okoth Bernard Wycliffe – 23/08325"  
  },  
  "security": {  
    "key_size": 32,  
    "hash_algorithm": "sha256",
```

```

"encryption_mode": "CBC"
},
"paths": {
  "input_dir": "./data/input",
  "output_dir": "./data/output",
  "keys_dir": "./data/keys",
  "logs_dir": "./data/logs"
}
}

```

8.1.2 Key Storage Format

encryption_key.json

```

json
{
  "key_id": "key_001",
  "algorithm": "AES-256",
  "created_date": "2025-07-21T14:30:15Z",
  "key_fingerprint": "sha256:a1b2c3d4...",
  "encrypted_key": "base64_encoded_encrypted_key",
  "salt": "base64_encoded_salt"
}

```

8.1.3 Processing Log Format

processing_log.json

```

json
{
  "session_id": "session_20250721_143015",
  "operations": [

```

```
{
  "timestamp": "2025-07-21T14:30:15Z",
  "operation": "protect_document",
  "file_path": "./input/academic-transcript.pdf",
  "status": "success",
  "hash": "a1b2c3d4e5f6g7h8...",
  "processing_time": 1.23
}
]
```

8.2 Data Persistence Strategy

8.2.1 Key Management

- **Key Generation:** Create secure random keys using OS entropy
- **Key Storage:** Encrypt keys with master password (PBKDF2 derived)
- **Key Backup:** Optional key backup to secure location
- **Key Rotation:** Support for periodic key updates

8.2.2 Audit Trail

- **Operation Logging:** Record all protection/verification operations
- **Error Logging:** Detailed error information for debugging
- **Performance Metrics:** Track processing times and success rates
- **User Actions:** Log significant user interactions

8.3 File Organization Structure

data/

```
├── input/ # Original documents
├── output/ # Protected documents
├── keys/ # Encryption keys
|   ├── master.key # Master encryption key
|   ├── document_001.key
|   └── document_002.key
├── logs/ # System logs
|   ├── application.log
|   └── operations.log
```

```
| └─ errors.log
└─ config/ # Configuration files
    └─ settings.json
    └─ user_preferences.json
```

9. Security Design

9.1 Cryptographic Standards

9.1.1 Hash Functions

- **Primary:** SHA-256 (256-bit output)
- **Backup:** SHA-3-256 (for future compatibility)
- **Performance:** Optimized for file sizes up to 100MB
- **Salt Integration:** Random salt for hash strengthening

9.1.2 Encryption Standards

- **Algorithm:** AES-256 in CBC mode
- **Key Derivation:** PBKDF2 with 100,000 iterations
- **IV Generation:** Cryptographically secure random IV per operation
- **Padding:** PKCS7 padding for block alignment

9.1.3 Random Number Generation

python

```
import secrets
```

```
import os
```

```
def generate_secure_random(size: int) -> bytes:
```

```
    """Generate cryptographically secure random bytes"""
```

```
    return secrets.token_bytes(size)
```

```
def generate_iv() -> bytes:
```

```
    """Generate random initialization vector"""
```

```
    return os.urandom(16) # 128-bit IV for AES
```

9.2 Key Management Security

9.2.1 Key Lifecycle

Key Generation → Key Storage → Key Usage → Key Rotation → Key Destruction

↓↓↓↓↓

Secure Random Encrypted Secure RAM New Key Gen Secure Wipe
Generation Storage Loading Process Memory

9.2.2 Key Storage Security

- **Encryption at Rest:** All keys encrypted with master password
- **Access Control:** File system permissions restrict key access
- **Key Derivation:** PBKDF2 with high iteration count
- **Backup Strategy:** Secure key backup with versioning

9.2.3 Memory Security

```
python
class SecureString:
    """Secure string handling for sensitive data"""
    def __init__(self, data: str):
        self._data = bytearray(data.encode('utf-8'))
    def __del__(self):
        # Overwrite memory before deletion
        for i in range(len(self._data)):
            self._data[i] = 0
    def get_data(self) -> str:
        return bytes(self._data).decode('utf-8')
```

9.3 Attack Mitigation

9.3.1 Common Attack Vectors

- Attack Type Mitigation Strategy Implementation
- Brute Force Strong key derivation PBKDF2 100k iterations
- Rainbow Table Random salt per operation os.urandom(32)
- Timing Attack Constant-time comparisons secrets.compare_digest()
- Memory Dump Secure memory handling Zero memory after use
- File Analysis Steganographic hiding LSB/whitespace embedding/pdf metadata

9.3.2 Security Validation

```
python
```



```
def validate_security_parameters():
    """Validate all security parameters meet standards"""
    checks = {
        'key_size': lambda: config.KEY_SIZE >= 32,
        'iv_size': lambda: config.IV_SIZE >= 16,
        'pbkdf2_iterations': lambda: config.PBKDF2_ITER >= 100000,
        'hash_algorithm': lambda: config.HASH_ALGO in ['sha256', 'sha3-256']
    }
    for check_name, check_func in checks.items():
        if not check_func():
            raise SecurityError(f"Security check failed: {check_name}")
```

9.4 Compliance Considerations

9.4.1 Academic Standards

- **Guidelines:** Follow cryptographic recommendations
- **Academic Integrity:** Clearly document all algorithm sources
- **Educational Value:** Include security analysis in documentation
- **Best Practices:** Implement industry-standard security patterns

9.4.2 Export Control

- **Encryption Strength:** AES-256 within academic exemption limits
- **Open Source:** Use only publicly available algorithms
- **Documentation:** Clear technical documentation for review
- **Use:** Academic and educational purposes only

10. Implementation Plan

10.1 Development Phases

Phase 1: Foundation (Weeks 1-2)

Objectives: Establish core infrastructure and basic functionality

Week 1 Tasks:

- Setup development environment (PyCharm, Python 3.8+)
- Create project structure and directory layout
- Implement configuration management system
- Develop basic file handling utilities
- Create logging infrastructure

Week 1 Deliverables:

- Project skeleton with all directories
- Configuration system with JSON storage
- Basic file I/O operations working
- Logging system capturing all operations

Week 2 Tasks:

- Implement SHA-256 hash generation
- Create AES encryption/decryption module
- Develop secure key generation
- Build basic unit tests for crypto functions
- Integrate error handling

Week 2 Deliverables:

- Working hash generation for all file types
- AES-256 encryption fully functional
- Comprehensive unit test suite
- Error handling with informative messages

Phase 2: Core Features (Weeks 3-4)

Objectives: Implement steganography and document verification

Week 3 Tasks:

- Implement text steganography (whitespace method)/pdf metadata
- Develop image steganography (LSB method)
- Create steganographic capacity calculations
- Build data extraction functions
- Test steganography with various file types

Week 3 Deliverables:

- Text steganography hiding data in whitespace/pdf metadata
- Image steganography using LSB technique
- Capacity calculator for optimal data embedding
- Extraction functions with error detection

Week 4 Tasks:

- Implement document verification workflow
- Create verification result reporting
- Develop batch processing capabilities (future integration)
- Integrate all core modules
- Performance optimization

Week 4 Deliverables:

- Complete document verification system
- Detailed verification reporting
- Batch processing for multiple files
- Performance benchmarks documented

Phase 3: User Interface (Weeks 5-6)

Objectives: Develop intuitive GUI and user experience

Week 5 Tasks:

- Design main window layout
- Implement file selection interface
- Create progress indication system
- Build results display components
- Develop dialog windows

Week 5 Deliverables:

- Complete main application window
- File drag-and-drop functionality (future integration)
- Real-time progress indicators
- Professional-looking GUI components

Week 6 Tasks:

- Implement key management interface
- Create detailed results dialogs
- Add help system and tooltips
- Develop error message displays
- GUI testing and refinement

Week 6 Deliverables:

- User-friendly key management system
- Comprehensive help documentation
- Polished user interface with error handling
- GUI responsiveness optimized

Phase 4: Testing & Integration (Weeks 7-8)

Objectives: Comprehensive testing and system integration validation

Week 7 Tasks:

- Develop comprehensive test suites for all modules
- Implement integration testing framework
- Create performance benchmarking tools
- Execute security penetration testing
- Document test results and coverage metrics

Week 7 Deliverables:

- Unit test suite
- Integration tests validating end-to-end workflows
- Performance benchmarks meeting specified requirements
- Security test results documenting vulnerability assessments

Week 8 Tasks:

- Conduct user acceptance testing with sample documents
- Perform stress testing with large file sets
- Validate cross-platform compatibility
- Create deployment packages
- Finalize documentation and user manual

Week 8 Deliverables:

- User acceptance test results with feedback incorporation
- Stress test validation for production readiness
- Cross-platform compatibility verification
- Complete deployment package with installer
- Comprehensive user manual and technical documentation

10.2 Resource Allocation

Human Resources:

- **Primary Developer:** 40 hours/week (full-time student dedication)
- **Academic Supervisor:** 2 hours/week (guidance and review sessions)
- **Peer Reviewers:** 4 hours total (code review and testing assistance)

Technical Resources:

- **Development Environment:** Windows 10 with PyCharm Community Edition
- **Testing Environment:** Virtual machines for cross-platform validation
- **Version Control:** Git repository with regular commits
- **Documentation Tools:** Markdown

Hardware Requirements:

- **Development Machine:** Minimum 8GB RAM, 256GB SSD
- **Testing Hardware:** Various systems for compatibility testing
- **Backup Storage:** Cloud storage for code and documentation backup

10.3 Risk Management

Technical Risks:

- Risk Impact Probability Mitigation Strategy
- Cryptographic Implementation
- Use established libraries, extensive testing
- Performance Issues with Large Files

- Implement chunked processing, optimization
- Cross-platform Compatibility
- Early testing on multiple platforms
- Steganography Detection
- Use multiple embedding techniques

Project Risks:

- Risk Impact Probability Mitigation Strategy
- Schedule Delays High
- Medium Buffer time in schedule, prioritize core features
- Scope Creep Medium
- Clear requirements documentation, change control
- Technical Complexity High
- Low Incremental development, regular supervisor consultation
- Resource Constraints Medium
- Low Focus on essential features, scalable architecture

11. Testing Strategy

11.1 Testing Approach

Testing Philosophy: The testing strategy follows a comprehensive approach ensuring both functional correctness and security validation. Testing will be conducted at multiple levels: unit, integration, system, and user acceptance testing.

Testing Levels:

1. **Unit Testing:** Individual module validation
2. **Integration Testing:** Module interaction verification
3. **System Testing:** End-to-end functionality validation
4. **Security Testing:** Cryptographic and security feature validation
5. **Performance Testing:** Scalability and efficiency measurement
6. **User Acceptance Testing:** Real-world usage scenario validation

11.2 Test Categories

11.2.1 Functional Testing

Hash Generation Testing:

Encryption Testing:

python

```
class TestHashGeneration(unittest.TestCase):
```

```
def test_sha256_consistency(self):
```

```
    """Verify hash consistency across multiple runs"""
```

```
def test_large_file_handling(self):
    """Test hash generation for files >10MB"""
def test_different_file_types(self):
    """Validate hash generation for TXT, PDF, DOCX, images"""
def test_empty_file_handling(self):
    """Edge case: empty file hash generation"""
```

Steganography Testing:

11.2.2 Security Testing

Cryptographic Validation:

- Key strength verification (256-bit entropy)
- Hash collision resistance testing
- Encryption algorithm compliance validation
- Random number generation quality assessment

Attack Simulation:

- Brute force attack resistance
- Timing attack mitigation verification

python

```
class TestEncryption(unittest.TestCase):
def test_aes256_encryption_decryption(self):
    """Verify encryption/decryption round-trip"""
def test_key_generation_uniqueness(self):
    """Ensure generated keys are unique"""
def test_iv_randomness(self):
    """Validate IV generation randomness"""
def test_padding_correctness(self):
    """Verify PKCS7 padding implementation"""
```

python

```
class TestSteganography(unittest.TestCase):
def test_text_embedding_extraction(self):
    """Verify text steganography round-trip"""
def test_image_lsb_embedding(self):
```

```

"""Test LSB steganography in images"""

def test_capacity_calculations(self):
    """Validate capacity calculation accuracy"""

def test_data_integrity(self):
    """Ensure embedded data remains intact"""

    • Memory dump analysis protection
    • Steganography detection resistance

```

Security Test Cases:

11.2.3 Performance Testing

Performance Benchmarks:

- Processing time for various file sizes
- Memory usage under different loads
- Batch processing efficiency
- GUI responsiveness metrics

Performance Test Specifications:

Test Scenario	File Size	Expected Time	Memory Limit
Small Text File	<1KB	<0.1s	<10MB
Medium Document	1-10MB	<2s	<50MB
Large Document	10-50MB	-	-
Batch Processing	100 files	-	-

11.2.4 Usability Testing

User Experience Validation:

- New user learning curve assessment
- Error message clarity evaluation
- Interface intuitiveness testing

python

```

class TestSecurity(unittest.TestCase):

def test_key_entropy(self):
    """Validate key generation entropy"""

def test_timing_attack_resistance(self):
    """Ensure constant-time operations"""

```

```
def test_memory_cleanup(self):  
    """Verify sensitive data is cleared from memory"""
```

```
def test_file_permission_security(self):  
    """Validate secure file storage permissions"""
```

Help system effectiveness validation

Usability Test Scenarios:

1. First-time user document protection
2. Batch document verification
3. Error recovery handling
4. Key management operations

11.3 Test Data Management

Test Data Categories:

- **Synthetic Data:** Generated files with known properties
- **Real Documents:** Sample business documents with permission
- **Edge Cases:** Corrupted, empty, and malformed files
- **Security Data:** Files with known vulnerabilities

Test Data Organization:

11.4 Test Automation

Automated Test Framework:

```
test_data/  
├── synthetic/  
│   ├── text_files/  
│   ├── documents/  
│   └── images/  
├── real_samples/  
│   ├── business_docs/  
│   └── certificates/  
├── edge_cases/  
│   ├── corrupted/  
│   ├── empty/  
│   └── malformed/  
└── security/
```


|— known_attacks/
|— vulnerability_samples/

Continuous Integration:

- Automated test execution on code commits
- Coverage reporting and trend analysis
- Performance regression detection
- Security vulnerability scanning

12. Deployment Plan

12.1 Deployment Strategy

Deployment Approach: The system will be deployed as a standalone desktop application with self-contained dependencies, suitable for academic demonstration and real-world usage.

Deployment Targets:

1. **Primary:** Windows 10/11 desktop installation
2. **Secondary:** Portable application package
3. **Tertiary:** Cross-platform compatibility package

12.2 Installation Package

python

```
class TestSuite:
```

```
def __init__(self):
```

```
self.test_runner = unittest.TextTestRunner(verbosity=2)
```

```
self.test_modules = [
```

```
'test_hash_generation',
```

```
'test_encryption',
```

```
'test_steganography',
```

```
'test_security',
```

```
'test_performance',
```

```
'test_integration'
```

```
]
```

```
def run_all_tests(self):
```

```
    """Execute complete test suite"""
```

```
def generate_coverage_report(self):
```

```
"""Generate code coverage analysis"""
```

```
def run_security_tests(self):
```

```
"""Execute security-specific test suite"""
```

Package Components:

12.3 Installation Process

Automated Installation Steps:

1. System requirements verification
2. Directory structure creation
3. File extraction and placement
4. Configuration file initialization
5. Registry entry creation (Windows)
6. Desktop shortcut creation
7. File association setup
8. Initial configuration wizard

Installation Script:

DocumentIntegrityProtection_v1.0/

```
|— bin/
| |— main.exe # Main application executable
| |— python39.dll # Python runtime
| |— required_libraries/ # Bundled dependencies
|— data/
| |— config/ # Default configuration files
| |— templates/ # Document templates
| |— examples/ # Sample files for testing
|— docs/
| |— user_manual.pdf # Comprehensive user guide
| |— technical_reference.pdf # Technical documentation
| |— quick_start_guide.pdf # Getting started guide
|— install/
| |— setup.exe # Windows installer
| |— uninstall.exe # Uninstaller
| |— registry_setup.bat # Registry configuration
```

└─ README.txt # Installation instructions

12.4 System Requirements

Minimum Requirements:

- **Operating System:** Windows 10 (64-bit)
- **RAM:** 4GB minimum, 8GB recommended
- **Storage:** 500MB available disk space
- **Processor:** Intel Core i3 or equivalent
- **Graphics:** DirectX 11 compatible
- **Network:** Internet connection for initial setup (optional)

Recommended Requirements:

- **Operating System:** Windows 11 (64-bit)
- **RAM:** 16GB for optimal performance
- **Storage:** 2GB available disk space
- **Processor:** Intel Core i5 or higher

```
python
```

```
class Installer:
```

```
def __init__(self):
```

```
self.install_path = self.get_install_path()
```

```
self.requirements = self.check_requirements()
```

```
def install(self):
```

```
    """Execute complete installation process"""
```

```
    steps = [
```

```
        self.verify_requirements,
```

```
        self.create_directories,
```

```
        self.extract_files,
```

```
        self.configure_system,
```

```
        self.create_shortcuts,
```

```
        self.register_file_types,
```

```
        self.run_initial_setup
```

```
    ]
```

```
    for step in steps:
```

```
        try:
```

```
step()
```

```
except Exception as e:
```

```
self.handle_installation_error(e)
```

```
return False
```

```
return True
```

- **Graphics:** Dedicated graphics card
- **Network:** Broadband internet connection

12.5 Configuration Management

Default Configuration:

Configuration Wizard:

- Initial security settings selection
- Default file locations setup
- User preference configuration
- Performance optimization options
- Network and update settings

12.6 Update Mechanism

Update Strategy:

```
json
```

```
{
```

```
"application": {
```

```
"version": "1.0.0",
```

```
"installation_date": "2025-07-21",
```

```
"user_preferences": {
```

```
"default_key_size": 32,
```

```
"auto_backup": true,
```

```
"theme": "default",
```

```
"language": "en"
```

```
}
```

```
},
```

```
"security": {
```

```
"hash_algorithm": "sha256",
```

```

"encryption_mode": "CBC",
"key_derivation_iterations": 100000,
"secure_delete": true
},
"performance": {
"max_memory_usage": 512,
"chunk_size": 1024,
"parallel_processing": true,
"cache_enabled": true
}
}

```

- Automatic update checking (optional)
- Manual update download and installation
- Configuration preservation during updates
- Rollback capability for failed updates

Update Process:

1. Version compatibility check
2. Backup current configuration
3. Download update package
4. Verify package integrity
5. Install update components
6. Migrate configuration settings
7. Validate installation
8. Clean up temporary files

12.7 Uninstallation

Clean Uninstallation Process:

- Application file removal
- Configuration data cleanup
- Registry entry removal
- Temporary file cleanup
- User data preservation option
- Shortcut and association cleanup

Uninstaller Features:

python

```

class Uninstaller:

    def __init__(self):

        self.install_info = self.load_installation_info()

    def uninstall(self, preserve_user_data=True):

        """Execute clean uninstallation"""

        tasks = [

            self.backup_user_data if preserve_user_data else None,

            self.remove_application_files,

            self.clean_registry_entries,

            self.remove_shortcuts,

            self.clear_file_associations,

            self.cleanup_temp_files

        ]

        for task in filter(None, tasks):

            task()

```

Appendices

Appendix A: Glossary

- **AES (Advanced Encryption Standard):** Symmetric encryption algorithm adopted by the U.S. government, using 256-bit keys for maximum security.
- **CBC (Cipher Block Chaining):** Block cipher mode of operation that chains each block of plaintext with the previous ciphertext block.
- **Hash Function:** Mathematical function that converts input data into a fixed-size string, used for integrity verification.
- **LSB (Least Significant Bit):** Steganographic technique that replaces the least significant bits of data with hidden information.
- **PBKDF2:** Password-Based Key Derivation Function 2, used to derive cryptographic keys from passwords.
- **SHA-256:** Secure Hash Algorithm producing 256-bit hash values, part of the SHA-2 family.
- **Steganography:** Practice of concealing information within other non-secret data or physical media.

Appendix B: References

1. NIST Special Publication 800-38A: "Recommendation for Block Cipher Modes of Operation"
2. RFC 2898: "PKCS #5: Password-Based Cryptography Specification Version 2.0"
3. FIPS PUB 180-4: "Secure Hash Standard (SHS)"
4. ISO/IEC 18033-3: "Encryption algorithms - Block ciphers"
5. Academic Papers on Steganography Techniques
6. Python Cryptography Library Documentation
7. Tkinter GUI Development Best Practices

Appendix C: Code Standards

Python Style Guide:

- Follow PEP 8 style guidelines
- Use type hints for all function signatures
- Maintain comprehensive documentations
- Implement proper error handling
- Use meaningful variable names
- Security Coding Standards:
- Never store sensitive data in plaintext
- Use secure random number generation
- Implement proper memory cleanup
- Validate all input parameters
- Use constant-time comparison operations

Appendix D: Academic Compliance

Project Requirements Fulfillment:

- ✓ Cryptographic implementation (SHA-256, AES-256)
- ✓ GUI development (Tkinter-based interface)
- ✓ File handling and processing
- ✓ Error handling and logging
- ✓ Documentation and testing
- ✓ Academic presentation preparation

Learning Objectives Achievement:

- Understanding of cryptographic principles
- Practical application of security concepts
- Software engineering best practices
- Project management experience
- Technical communication skills

Document Version: 1.0. Last Updated: July 21, 2025